

Table 1: End-to-end execution time (s, median per iteration) and speedup split by pass type. When present, the OctTree row includes ColorOctree passes; a separate ColorOctree row reports only those passes. ADT fields = non-recursive fields annotated with @BENCH adt_fields; SoA bufs = 1+ non-recursive field slots across all constructors (recursive children are stored in the tag buffer). Speedup $>1\times$ means SoA is faster; **bold** marks $>1.1\times$.

Program	ADT	SoA	Fold passes			Map passes		
	fields	bufs	AoS (s)	SoA (s)	Speedup	AoS (s)	SoA (s)	Speedup
OctTree	16	9	0.3671	0.3268	1.12 $\times$	0.3033	0.3515	0.86 $\times$
Compiler	9	8	0.1413	0.0551	2.56 $\times$	0.1647	0.1807	0.91 $\times$
DBQuery	15	13	0.1226	0.0882	1.39 $\times$	0.1423	0.1667	0.85 $\times$
DecisionTree	7	6	0.5637	0.6311	0.89 $\times$	–	–	–
DomTree	15	14	0.1410	0.0651	2.16 $\times$	0.0588	0.0822	0.72 $\times$
KDTree	17	16	0.1794	0.1391	1.29 $\times$	–	–	–
LinearListReduction	11	11	0.0301	0.0051	5.87 $\times$	–	–	–
List	2	2	0.3237	0.3084	1.05 $\times$	0.2521	0.3386	0.74 $\times$
MonoTree	3	2	0.0175	0.0223	0.79 $\times$	0.0571	0.0423	1.35 $\times$
ObjectGraph	5	4	0.0974	0.0797	1.22 $\times$	0.1452	0.1612	0.90 $\times$
PiecewiseFunctions	8	7	0.1380	0.0911	1.52 $\times$	0.2361	0.2629	0.90 $\times$
TernaryTree	5	3	0.0301	0.0286	1.05 $\times$	0.0838	0.0794	1.06 $\times$
Trie	10	9	0.0652	0.0315	2.07 $\times$	0.1517	0.1702	0.89 $\times$
ColorOctree	25	18	0.0889	0.0761	1.17 $\times$	–	–	–

Table 2: Mutable vs immutable cursor comparison (times only). Times are median per iteration (s). **Bold** marks the fastest time across all four variants.

Program	Am	Ai	Sm	Si
Compiler	0.3060	–	0.2358	–
DBQuery	0.2649	–	0.2549	–
DecisionTree	0.5637	–	0.6311	–
DomTree	0.1998	–	0.1473	–
KDTree	0.1794	–	0.1391	–
LinearListReduction	0.0301	–	0.0051	–
List	0.5758	–	0.6469	–
MonoTree	0.0747	–	0.0646	–
ObjectGraph	0.2426	–	0.2408	–
OctTree_sumMass	0.0338	–	0.0270	–
OctTree_sumEnergy	0.0369	–	0.0331	–
OctTree_countActive	0.0354	–	0.0291	–
OctTree_countParticles	0.0351	–	0.0269	–
OctTree_barnesHutPotential	0.0410	–	0.0392	–
OctTree_fmmPotential	0.0960	–	0.0954	–
OctTree_scaleEnergy	0.1502	–	0.1741	–
OctTree_clearFlags	0.1531	–	0.1775	–
PiecewiseFunctions	0.3741	–	0.3540	–
TernaryTree	0.1139	–	0.1080	–
Trie	0.2168	–	0.2017	–
ColorOctree	0.0889	–	0.0761	–

Table 3: Mutable vs immutable cursor comparison (speedups). Shown: AoS-mut/SoA-mut, AoS-imm/AoS-mut, AoS-imm/SoA-mut, AoS-imm/SoA-imm. $>1\times$ means the denominator is faster.

Program	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
Compiler	1.30 $\times$	–	–	–
DBQuery	1.04 $\times$	–	–	–
DecisionTree	0.89 $\times$	–	–	–
DomTree	1.36 $\times$	–	–	–
KDTree	1.29 $\times$	–	–	–
LinearListReduction	5.87 $\times$	–	–	–
List	0.89 $\times$	–	–	–
MonoTree	1.16 $\times$	–	–	–
ObjectGraph	1.01 $\times$	–	–	–
OctTree_sumMass	1.25 $\times$	–	–	–
OctTree_sumEnergy	1.12 $\times$	–	–	–
OctTree_countActive	1.22 $\times$	–	–	–
OctTree_countParticles	1.30 $\times$	–	–	–
OctTree_barnesHutPotential	1.05 $\times$	–	–	–
OctTree_fmmPotential	1.01 $\times$	–	–	–
OctTree_scaleEnergy	0.86 $\times$	–	–	–
OctTree_clearFlags	0.86 $\times$	–	–	–
PiecewiseFunctions	1.06 $\times$	–	–	–
TernaryTree	1.05 $\times$	–	–	–
Trie	1.07 $\times$	–	–	–
ColorOctree	1.17 $\times$	–	–	–

Table 4: Runtime comparison of Gibbon (AoS/SoA), GHC, and MLton. Times are total median per iteration (s). **Bold** marks the fastest time for each program.

Program	Gibbon-AoS	Gibbon-SoA	GHC	MLton
Compiler	0.3060	0.2358	2.002	–
DBQuery	0.2649	0.2549	2.659	–
DecisionTree	0.5637	0.6311	4.010	–
DomTree	0.1998	0.1473	1.369	–
KDTree	0.1794	0.1391	1.588	–
LinearListReduction	0.0301	0.0051	1.266	–
List	0.5758	0.6469	16.95	–
MonoTree	0.0747	0.0646	1.517	–
ObjectGraph	0.2426	0.2408	2.275	–
OctTree_sumMass	0.0338	0.0270	3.337	–
OctTree_sumEnergy	0.0369	0.0331	5.864	–
OctTree_countActive	0.0354	0.0291	5.254	–
OctTree_countParticles	0.0351	0.0269	5.396	–
OctTree_barnesHutPotential	0.0410	0.0392	5.307	–
OctTree_fmmPotential	0.0960	0.0954	5.781	–
OctTree_scaleEnergy	0.1502	0.1741	5.893	–
OctTree_clearFlags	0.1531	0.1775	5.301	–
PiecewiseFunctions	0.3741	0.3540	3.001	–
TernaryTree	0.1139	0.1080	1.007	–
Trie	0.2168	0.2017	1.698	–
ColorOctree	0.0889	0.0761	7.073	–
Geomean	0.1214	0.1031	3.276	–

Table 5: Gibbon mutable speedups vs GHC. Each entry is total median runtime speedup over all passes for one iteration. *AoS/GHC* and *SoA/GHC* are reported.

Program	AoS/GHC	SoA/GHC
Compiler	0.15×	0.12×
DBQuery	0.10×	0.10×
DecisionTree	0.14×	0.16×
DomTree	0.15×	0.11×
KDTree	0.11×	0.09×
LinearListReduction	0.02×	0.00×
List	0.03×	0.04×
MonoTree	0.05×	0.04×
ObjectGraph	0.11×	0.11×
OctTree_sumMass	0.01×	0.01×
OctTree_sumEnergy	0.01×	0.01×
OctTree_countActive	0.01×	0.01×
OctTree_countParticles	0.01×	0.00×
OctTree_barnesHutPotential	0.01×	0.01×
OctTree_fmmPotential	0.02×	0.02×
OctTree_scaleEnergy	0.03×	0.03×
OctTree_clearFlags	0.03×	0.03×
PiecewiseFunctions	0.12×	0.12×
TernaryTree	0.11×	0.11×
Trie	0.13×	0.12×
ColorOctree	0.01×	0.01×
Geomean	0.04×	0.03×

Table 6: Per-pass performance for **OctTree**, OctTree SoA uses 9 buffers; ColorOctree SoA uses 18 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
barnesHutPotential	F	11/16	31%	0.0410	0.0392	1.05 $\times$
clearFlags	M	15/16	6%	0.1531	0.1775	0.86 $\times$
countActive	F	10/16	38%	0.0354	0.0291	1.22$\times$
countParticles	F	8/16	50%	0.0351	0.0269	1.30$\times$
fmmPotential	F	12/16	25%	0.0960	0.0954	1.01 $\times$
scaleEnergy	M	16/16	0%	0.1502	0.1741	0.86 $\times$
sumEnergy	F	12/16	25%	0.0369	0.0331	1.12$\times$
sumMass	F	10/16	38%	0.0338	0.0270	1.25$\times$
paletteEntriesQuantized	F	13/25	48%	0.0429	0.0334	1.28$\times$
quantizationErrorProxy	F	10/25	60%	0.0460	0.0427	1.08 $\times$
Total				0.6703	0.6783	0.99 $\times$
Geomean						1.09 $\times$

Table 7: Per-pass performance for **Compiler**, ADT has 9 fields, SoA uses 8 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
blockCountPass	F	2/9	78%	0.0125	0.0019	6.46$\times$
branchStatsPass	F	2/9	78%	0.0120	0.0035	3.46$\times$
castInstCountPass	F	3/9	67%	0.0188	0.0013	14.15$\times$
has cycle	F	4/9	56%	0.0356	0.0330	1.08 $\times$
instCountPass	F	2/9	78%	0.0125	0.0020	6.19$\times$
latencyModelPass	F	3/9	67%	0.0124	0.0030	4.15$\times$
memoryOpStatsPass	F	3/9	67%	0.0122	0.0038	3.20$\times$
stripSideEffectsPass	M	7/9	22%	0.0825	0.0903	0.91 $\times$
targetReturnPass	M	9/9	0%	0.0822	0.0904	0.91 $\times$
throughputModelPass	F	3/9	67%	0.0124	0.0030	4.20$\times$
verifyIR	F	9/9	0%	0.0129	0.0036	3.62$\times$
Total				0.3060	0.2358	1.30$\times$
Geomean						3.19$\times$

Table 8: Per-pass performance for **DBQuery**, ADT has 15 fields, SoA uses 13 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
clearQueryFlags	M	14/15	7%	0.0706	0.0828	0.85 $\times$
countJoins	F	3/15	80%	0.0191	0.0113	1.68$\times$
filterSelectivitySkew	F	5/15	67%	0.0190	0.0125	1.52$\times$
hashJoinPressure	F	5/15	67%	0.0207	0.0131	1.58$\times$
scaleCosts	M	15/15	0%	0.0717	0.0840	0.85 $\times$
sumCost	F	6/15	60%	0.0191	0.0150	1.27$\times$
sumMemory	F	6/15	60%	0.0193	0.0147	1.31$\times$
sumRows	F	6/15	60%	0.0256	0.0216	1.18$\times$
Total				0.2649	0.2549	1.04 $\times$
Geomean						1.25$\times$

Table 9: Per-pass performance for **DecisionTree**, ADT has 7 fields, SoA uses 6 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
classify Batch	F	5/7	29%	2.00e-05	7.20e-05	0.28 $\times$
classify tree	F	5/7	29%	–	3.00e-06	–
countFeatureUses	F	3/7	57%	0.0639	0.0738	0.87 $\times$
countLeaves	F	2/7	71%	0.0612	0.0658	0.93 $\times$
countNodes	F	2/7	71%	0.0618	0.0658	0.94 $\times$
countSmallLeaves	F	3/7	57%	0.0641	0.0720	0.89 $\times$
inferenceCost	F	2/7	71%	0.0615	0.0664	0.93 $\times$
sumImpurity	F	3/7	57%	0.0631	0.0719	0.88 $\times$
sumPathLengths	F	3/7	57%	0.0630	0.0755	0.83 $\times$
sumSamples	F	3/7	57%	0.0625	0.0733	0.85 $\times$
treeDepth	F	2/7	71%	0.0625	0.0664	0.94 $\times$
Total				0.5637	0.6311	0.89 $\times$
Geomean						0.80 $\times$

Table 10: Per-pass performance for **DomTree**, ADT has 15 fields, SoA uses 14 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
SumArea	F	6/15	60%	0.0361	0.0186	1.95$\times$
computeWidths	M	14/15	7%	0.0297	0.0462	0.64 $\times$
count styled	F	3/15	80%	0.0349	0.0115	3.03$\times$
find max Bottom	F	5/15	67%	0.0349	0.0233	1.50$\times$
scaleLayout	M	15/15	0%	0.0291	0.0359	0.81 $\times$
sumTextWidth	F	3/15	80%	0.0351	0.0117	2.99$\times$
Total				0.1998	0.1473	1.36$\times$
Geomean						1.55$\times$

Table 18: Per-pass performance for **Trie**, ADT has 10 fields, SoA uses 9 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across $-iterate$ runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
autocompleteTopKProxy	F	5/10	50%	0.0146	0.0079	1.86 $\times$
countLazyNodes	F	4/10	60%	0.0131	0.0066	1.98 $\times$
countTerminals	F	3/10	70%	0.0124	0.0053	2.36 $\times$
decayTrieStats	M	10/10	0%	0.0761	0.0867	0.88 $\times$
resetTraversalState	M	10/10	0%	0.0755	0.0835	0.90 $\times$
sumPrefixFreq	F	3/10	70%	0.0125	0.0058	2.14 $\times$
sumSubtreeHints	F	3/10	70%	0.0125	0.0059	2.12 $\times$
Total				0.2168	0.2017	1.07 $\times$
Geomean						1.64 $\times$